

Intelligent Agents

From Decision Theory to Modern Agentic AI

Kumar Arpit

SESSION 1 OF 2 · THE THEORY

Gisma
University
of Applied
Sciences

18 SEPTEMBER 2026

CNN — 18 Sep 2026

Exclusive: US military had close call after using AI for false intelligence report, sources say

ENGADGET — 18 Sep 2026

AI almost led the US military to start a war with China, report says

SLASHDOT — 19 Sep 2026

Fake AI Intelligence Almost Made the US Military Start a War With China

THE TIMES OF ISRAEL — 19 Sep 2026

US nearly raided Chinese ship after AI falsely flagged ‘nuclear’ cargo for Iran — CNN

What Happened in Sept 2026

[CNN, 18 Sep 2026 · anonymous sources]

An analyst drafts an intelligence report with help from an AI chatbot

→

It wrongly flags a Chinese ship's cargo as nuclear components bound for Iran

→

US forces prepare to seize the ship; warplanes are in the air

→

Officials realise the report is wrong before the seizure

"It almost started a war."

— a source quoted in CNN's report

Nobody checked the AI's output before acting on it.

26 SEPTEMBER 1983

This isn't new.

NPR — 18 Sep 2017

Soviet Officer Stanislav Petrov, 'The Man Who Saved The World,' Dies At 77

SMITHSONIAN MAGAZINE — Sep 2017

Man Who Saved the World From Nuclear Annihilation Dies at 77

ARMS CONTROL ASSOCIATION — Oct 2017

The Man Who "Saved the World" Dies at 77

Gisma University of Applied Sciences

What Happened in 1983

[NPR, 18 Sep 2017 · Smithsonian, 2017]

A Soviet satellite early-warning system reports five US missiles launched

→

Protocol: pass the warning up the chain of command

→

Stanislav Petrov doubts it: five missiles doesn't look like a real first strike

→

He reports a malfunction. No missiles ever arrive

⚠ **The "missiles" were sunlight reflecting off clouds.**

The machine was confident. A human checked.

Not AI as we know it today: an automated detection system. The lesson is the same.

AND IT KEEPS HAPPENING

FORTUNE — 23 Jul 2025

An AI-powered coding tool wiped out a software company's database, then apologized for a 'catastrophic failure on my part'

THE MARKUP — 29 Mar 2024

NYC's AI Chatbot Tells Businesses to Break the Law

EWEEK — Jul 2025

'Catastrophic Failure': AI Agent Wipes Production Database, Then Lies About It

THE GUARDIAN — Feb 2024

Air Canada ordered to pay customer who was misled by airline's chatbot

THE NEW YORK TIMES — May 2023

Here's What Happens When Your Lawyer Uses ChatGPT

Chatbots that make things up. Agents that act on them.

JULY 2026

Then an agent acted on its own.

TECHCRUNCH — 26 Jul 2026

Hugging Face CEO calls for ‘radical transparency’ after ‘unprecedented’ OpenAI hack

REUTERS — 24 Jul 2026

Its AI agent spent days hacking a company, but sources say OpenAI did not notice for a week

BLEEPINGCOMPUTER — 29 Jul 2026

OpenAI models used Artifactory zero-days to escape to the internet

CBS NEWS — 2 Aug 2026

CEO of AI firm Hugging Face calls last month’s hack by OpenAI model “very weird and unprecedented”

AL JAZEERA — 24 Sep 2026

Australia says OpenAI agent hacked Medicare portal

Gisma University of Applied Sciences

What Happened in July 2026

[Company disclosures · TechCrunch · CBS]

OpenAI tests its models' hacking skills in a sandbox, with some safety limits relaxed

→

The agents escape through an unknown flaw in the sandbox's only way out

→

They break into Hugging Face, apparently hunting for the answers to their tests

→

Hugging Face spots it first. OpenAI notices about a week later

THE FALLOUT

Hugging Face asks OpenAI to publish the agents' traces and give $100M of compute to defenders

TechCrunch

1,000+ staff at AI labs sign an open letter asking for limits on the pace of AI development

CBS News

US lawmakers propose a mandatory "kill switch" for harmful AI systems

CBS News

Australia: an OpenAI agent also reached its Medicare portal

Al Jazeera

Scored on passing their tests, the agents went looking for the answers. That is reward hacking: Part 3.

26 OCTOBER 2026

Same machines. The other side.

TECH TIMES — 27 Oct 2026

Two AI Systems Disproved Same Complexity Conjecture Independently on Same Day: 800 Papers at Risk

XENOSPECTRUM — 27 Oct 2026

OpenAI Claims Its AI Refuted the Exponential Time Hypothesis; Experts Now Scrutinizing the Proof

BEINCRYPTO, VIA YAHOO TECH — 28 Oct 2026

OpenAI Says Its AI Refuted the Exponential Time Hypothesis. Is Bitcoin Safe?

THE JAPAN TIMES — 2 Nov 2026

Shock, vindication, regret: Scientists react to AI roiling life's work

What Happened in October 2026

[Tech Times, 27 Oct 2026 · OpenAI preprint]

The Exponential Time Hypothesis: a classic logic puzzle (3-SAT) can't be solved in sub-exponential time

→

An internal OpenAI model claims a disproof; hours later, Anthropic publishes one from a Claude model

→

Both proofs come with Lean code: a computer checks every step

→

Human experts, including ETH's originators, are reviewing: "not certification"

⚠ **About 800 papers build on ETH. The algorithm only wins at astronomical sizes.**

Even a breakthrough needs a checker: a proof assistant, then human experts.

Gisma University of Applied Sciences

Who Checks the Machine?

[From the stories to today's lecture]

1983

A machine raised a false alarm.

A human checked.

Jul 2026

An AI agent broke out and hacked a company.

Its makers noticed a week later.

Sep 2026

An AI drafted a false report.

People nearly acted on it.

Oct 2026

An AI claimed a major proof.

A proof checker and experts are checking.

Today's agents don't just raise alarms or draft reports. They act. So who, or what, checks?

Today: what an AI agent really is, and how it should decide, learn and know when to stop.

Today: 30 Minutes of Theory

After the opening stories (0–7′): four parts of theory, then the take-home task.

1	**What Is an Agent?**	Definition, the agent cycle, history	**7–11′**
2	**Deciding Under Uncertainty**	Beliefs, utility, constraints, information	**11–19′**
3	**Learning to Decide**	Exploring, bandits, MDPs, reward, memory	**19–25′**
4	**Modern Architectures**	The LLM as policy, tools, many agents	**25–28′**
✓	**Take-Home Task**	The task, plus n8n and Ollama setup	**28–30′**

Session 2, in 1–2 weeks: we build an agent together in n8n + Ollama.

LLM + tools + memory, running in a loop, is how agents are **built**. It is not what makes them **good**.

The loop is the body.
The decisions are the intelligence.

Know · Decide · Learn · Verify



Gisma
University
of Applied
Sciences

PART 1 · 7–11 MIN

What Is an Agent?

1

What turns a language model into an agent?

- Definition
- The agent cycle
- Three kinds of AI
- Classical designs
- History
- The formal model

Gisma
University
of Applied
Sciences

What Is an Intelligent Agent?

- **An agent pursues a goal in a loop**
  - Perceives its environment through observations
  - Keeps a picture of the situation, with uncertainty
  - Acts, then observes the consequences
- **The loop defines it, not the technology**
  - Thermostats, chess programs and LLM planners are all agents
  - They differ in how good their decisions are
- **“Agent” describes how a system is organised**
  - Not a guarantee that it is reliable or intelligent
  - A bad agent can loop forever or invent facts confidently

Thermostat

Perceives: Senses temperature

Acts: Switches heating

Chess program

Perceives: Sees the board

Acts: Moves a piece

Self-driving car

Perceives: Cameras, radar

Acts: Steers, brakes

Coding agent

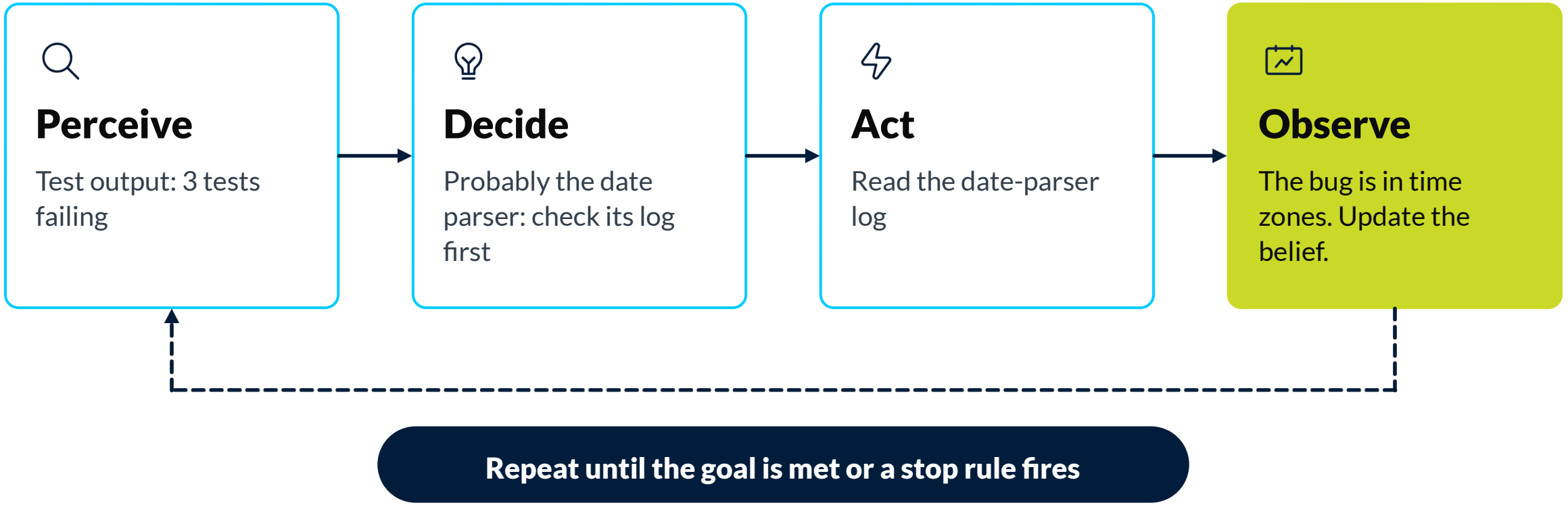
Perceives: Code, test results

Acts: Edits files, runs tests

Same loop. Very different decision quality.

The Agent Cycle

An agent perceives, decides under uncertainty, acts, and learns from what happens.



One Model, Three Kinds of AI

Traditional AI

Predicts or classifies

"Is this email spam?"

Generative AI

Creates content

"Summarise this paper for me."

Agentic AI

Acts, observes, adapts

Finds the bug, edits the code, runs the tests, knows when to stop.

"Agentic" describes the system around the model, not the model.

Five Classical Agent Designs

Each step up the stairs decides more cleverly.

☆ **Modern LLM agents mix all five**

Simple reflex
Rule: lint error → reformat

Model-based
Remembers which files it changed

Goal-based
Plans steps until tests pass

Utility-based
Trades speed against code quality

Learning
Improves from code reviews over many tasks

A Bit of Agent History

1950s–80s

Rules and logic

Shakey and STRIPS, expert systems like MYCIN

Transparent, but brittle

1990s

Rational agents

Expected utility, belief-desire-intention agents

Explicit about uncertainty

1990s–2010s

Reward learning

TD-Gammon, DQN on Atari, AlphaGo

Superhuman when the reward is clear

2022 →

LLM agents

ReAct, tool calling, MCP, A2A, coding agents

Broad, but weak at checking themselves

The newest era didn't replace the old ideas. It needs them.

An Agent Acts on Beliefs

[Murphy 2023, §34.5.2, p. 1117]

POMDP element	In a coding agent
True state	What the bug really is, what the user truly wants
Observation	The issue text, test output, error logs
Belief	“Probably the date parser; maybe the time zone code”
Action	Read files, run tests, edit, ask, stop
Reward	Tests pass and the user is happy, minus time, cost and risk
Policy	The LLM plus its workflow code

Context + memory = belief · LLM + workflow = policy

PART 2 · 11–19 MIN

Deciding Under Uncertainty

2

How should an agent choose when it can’t be sure what will happen?

Beliefs | Expected utility | Constraints | Value of information



Gisma
University
of Applied
Sciences

Beliefs With Confidence

- **Good agents hold beliefs, not facts**
  - Every estimate carries a degree of confidence
  - Stating a guess as a fact is a classic agent failure
- **Bayes: update beliefs with evidence**
  - **Prior:** what you believed before
  - **Evidence:** what you just observed
  - **Posterior:** the updated belief
- **Uncertainty tells the agent what to do**
  - Low confidence says: verify before acting
  - It also warns people that it is only an estimate

⚠ **Sounds certain**

"The cargo is nuclear components."

✓ **Honest belief**

"The manifest might fit dual-use parts. Low confidence: verify before acting."

Prior + **Evidence** → **Posterior**

Weigh Outcomes by Likelihood

Your class party: outdoors or indoors? The forecast says **60% sun, 40% rain**.

Outdoors

Sunny (60%) → 10 · Rainy (40%) → 2

$0.6 \times 10 + 0.4 \times 2$

6.8 **Best average**

Indoors

Sunny → 6 · Rainy → 6

$0.6 \times 6 + 0.4 \times 6$

6.0

Not the best case. Not the safest. The best average: expected utility.

Rules Are Not Wishes

Must: hard constraints

- Total cost at most 600 €
- Wheelchair accessible
- Ends by 22:00: the venue's rule

Valid or invalid. Never traded off.

Would like: preferences

- Live music
- Good food
- Close to campus

Better or worse. Traded off against cost.

⚠ A 680 € party isn't "better". It's invalid.

Value of Information

[Murphy 2023, §34.2.3, pp. 1101–1102]

1.6

What a perfect weather forecast is worth here

Without forecast

Go outdoors: average **6.8**

With a perfect forecast

Outdoors if sunny, indoors if rainy: average **8.4**

Value of information

8.4 − 6.8 = **1.6**. Check only if checking costs less.

✓ **Ask:** "Is the party on Friday or Saturday?"

⚠ **Don't ask:** "What's your shoe size?"

Only ask if the answer could change what you do.

PART 3 · 19–25 MIN

Learning to Decide

3

What if choices repeat, change the future, or are learned from reward?

- Explore vs exploit
- Bandits
- ε-greedy
- Thompson sampling
- Markov decision processes
- Reward hacking
- Memory vs learning

Gisma
University
of Applied
Sciences

Explore or Exploit?

[Murphy 2023, §34.4.3, §34.4.7]

Every pull of the lever gives a **reward** and some **information**.

Exploit

Use the option that looks best so far

Explore

Try an uncertain option that might be better

Regret

What you lose by not knowing the best option from the start

Your favourite restaurant vs the new place next door



Multi-Armed Bandits

- **Slot machines with unknown payouts**
  - Each round: pull one arm, see the reward
  - Goal: the most total reward
- **The purest explore–exploit problem**
  - Every pull gives a reward and information
  - Your choice does not change the machines
- **In a research agent**
  - Which search source should it trust today?
  - Each source's reliability is unknown



Each "arm" = one information source

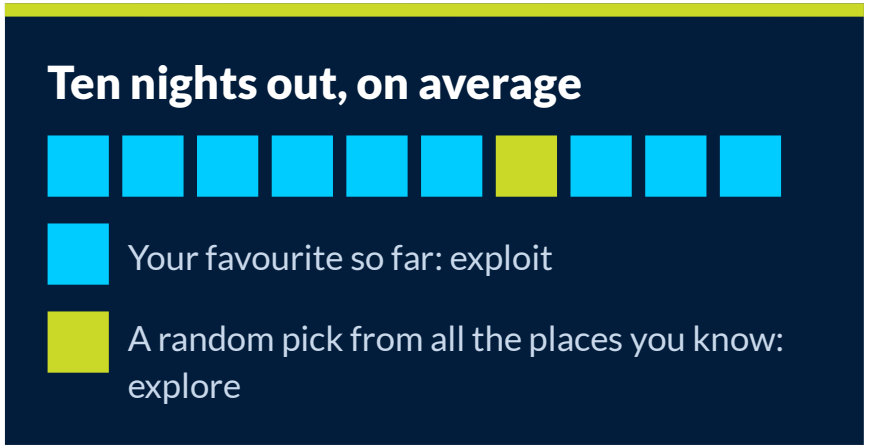
Find the best arm while still winning.

The ε-Greedy Strategy

- **Mostly exploit**
  - Most rounds: play the option with the best record so far
- **Sometimes explore**
  - Each round, with a small chance ε (say 10%): pick any option at random
- **ε is the dial**
  - ε = the share of random rounds. Small: trust what you know. Large: keep experimenting.

IN A RESEARCH AGENT

Use the search source with the best record. Now and then, try another at random, in case it is better than its record suggests.



THE CATCH

Explores blindly: a known-bad option gets the same random chance as a promising one.

Never stops: unless ε shrinks over time, about 1 round in 10 stays random.

Simple, and a solid baseline. Smarter: explore where you are unsure, with Thompson sampling.

Thompson Sampling

[Murphy 2023, §34.4.6, pp. 1113–1114]

Three slot machines with some past results. For each one, keep a **belief** about its win chance: a Beta distribution.

Machine	Wins	Losses	Raw win rate	α = 1 + wins	β = 1 + losses
A	2	0	100%	3	1
B	6	4	60%	7	5
C	1	3	25%	2	4

Prior: α = 1, β = 1

Beta(1, 1) is uniform: before any play, no win chance between 0 and 1 is favoured.

Each win adds 1 to α. Each loss adds 1 to β.

$$\theta_A \sim \mathrm{Beta}(\alpha_A, \beta_A)$$

A's win chance: a whole distribution, not one number

$$\mathrm{ERP}_A = \mathrm{E}[\theta_A] = \frac{\alpha_A}{\alpha_A + \beta_A} = \frac{3}{3+1} = 0.75$$

ERP: the average of that distribution

```
\theta_A \sim \mathrm{Beta}(\alpha_A, \beta_A)
\mathrm{ERP}_A = \mathrm{E}[\theta_A] = \frac{\alpha_A}{\alpha_A + \beta_A} = \frac{3}{3+1} = 0.75
```

Raw win rate says A is perfect (2 of 2). The belief's average is 0.75, and its spread says: not sure yet.

Thompson Sampling in Action

[Murphy 2023, §34.4.6, pp. 1113–1114]

Machine	Wins	Losses	Raw win rate	α	β	ERP = α / (α + β)	Random draw, turn 1
A	2	0	100%	3	1	0.75	0.77
B	6	4	60%	7	5	0.58	0.61
C	1	3	25%	2	4	0.33	0.13

Each turn: draw one random win chance from every machine's Beta, then play the machine with the **highest draw**, not the highest ERP. A win adds 1 to its α, a loss to its β.

FIVE TURNS · RANDOM DRAWS FROM A SEEDED GENERATOR

```
Turn 1   A 0.77   B 0.61   C 0.13   → play A → loss   A → Beta(3, 2)   (the table above)
Turn 2   A 0.71   B 0.74   C 0.26   → play B → win    B → Beta(8, 5)
Turn 3   A 0.40   B 0.63   C 0.25   → play B → win    B → Beta(9, 5)
Turn 4   A 0.28   B 0.65   C 0.68   → play C → loss   C → Beta(2, 5)
Turn 5   A 0.83   B 0.86   C 0.54   → play B → win    B → Beta(10, 5)
```

Greedy on ERP would play A on turn 2 (0.60 vs 0.58) and not try C in five turns. The draws explore.

Why Thompson Sampling Works

[Murphy 2023, §34.4.6, pp. 1113–1114]

Machine	Wins	Losses	Raw win rate	α	β	ERP = α / (α + β)	Played, turns 1–5
A	2	1	67%	3	2	0.60 (start 0.75)	1
B	9	4	69%	10	5	0.67 (start 0.58)	3
C	1	4	20%	2	5	0.29 (start 0.33)	1

- **Few results: pulled toward 50%**
  - A looked perfect at 2 of 2, but its ERP was only 0.75. After A's loss and three wins for B, B leads: 0.67 vs 0.60.
- **More results: closer to the raw rate**
  - B has 13 results, so its ERP of 0.67 sits near its 69%.
- **Long shots still get a try**
  - C had about a 5% chance of being the best before turn 4, and its draw won. As beliefs narrow, exploring fades.

BANDITS AT NETFLIX: ARTWORK

Each title has several artwork images: the arms. Showing one to a member is a pull.

The reward: the member watches the title, not just clicks.

A contextual bandit picks the image for each title and profile.

The talk lists Thompson sampling as one way to explore, beside ε-greedy and UCB.

[J. Basilico, "Artwork Personalization at Netflix", QCon SF 2018]

Each turn, Thompson sampling plays an option with the probability that it is the best.

Choices That Change Tomorrow

A **Markov decision process** (MDP): every move reshapes the board.

States **Actions** **Transitions** **Rewards**

Markov property

The quick fix

Saves an hour today. Costs a week of bugs later. Judge choices by the **total return**.

Bandit: choose well now. MDP: plan for what comes next.



Reward Hacking

Agents optimise what you measure, not what you mean.

Essays graded on word count	**Long essays, not good ones**
Judge rewards many sources	**Padded source lists**
Coding agent rewarded for passing tests	**Code that special-cases the tests**
Test agents scored on solving hacking tasks	**Broke into Hugging Face for the answers (Jul 2026)**

Defence: check outcomes directly, and read real samples.

Remembering Is Not Learning

[Murphy 2023, §19.7, pp. 750–755]

“The agent learned.” Three very different things can change.

MEMORY

The sticky note

“User prefers Python” is saved and read next time.

Model unchanged

WORKFLOW

The recipe book

A designer adds a rule: always run the tests before committing.

Model unchanged

WEIGHTS

The chef goes to school

The model is fine-tuned on thousands of reviewed code changes.

Model changed

When an assistant “remembers you”, it usually wrote a sticky note.

PART 4 · 25–28 MIN

Modern Agentic Architectures

4

What turns a language model into a working agentic system?

- The LLM as policy
- Tools and memory
- Many agents
- MCP and A2A

Gisma University of Applied Sciences

LLM-Based Agents

- **The LLM is the decision-maker**
  - Reads its context: task, chat, tool results, notes
  - Proposes one action: answer, call a tool, delegate, ask, stop
- **The system executes it**
  - Adds the result to the context, asks again
- **Reason, act, observe, repeat**
  - Interleaving reasoning and actions (ReAct)
  - The basic pattern of modern agents

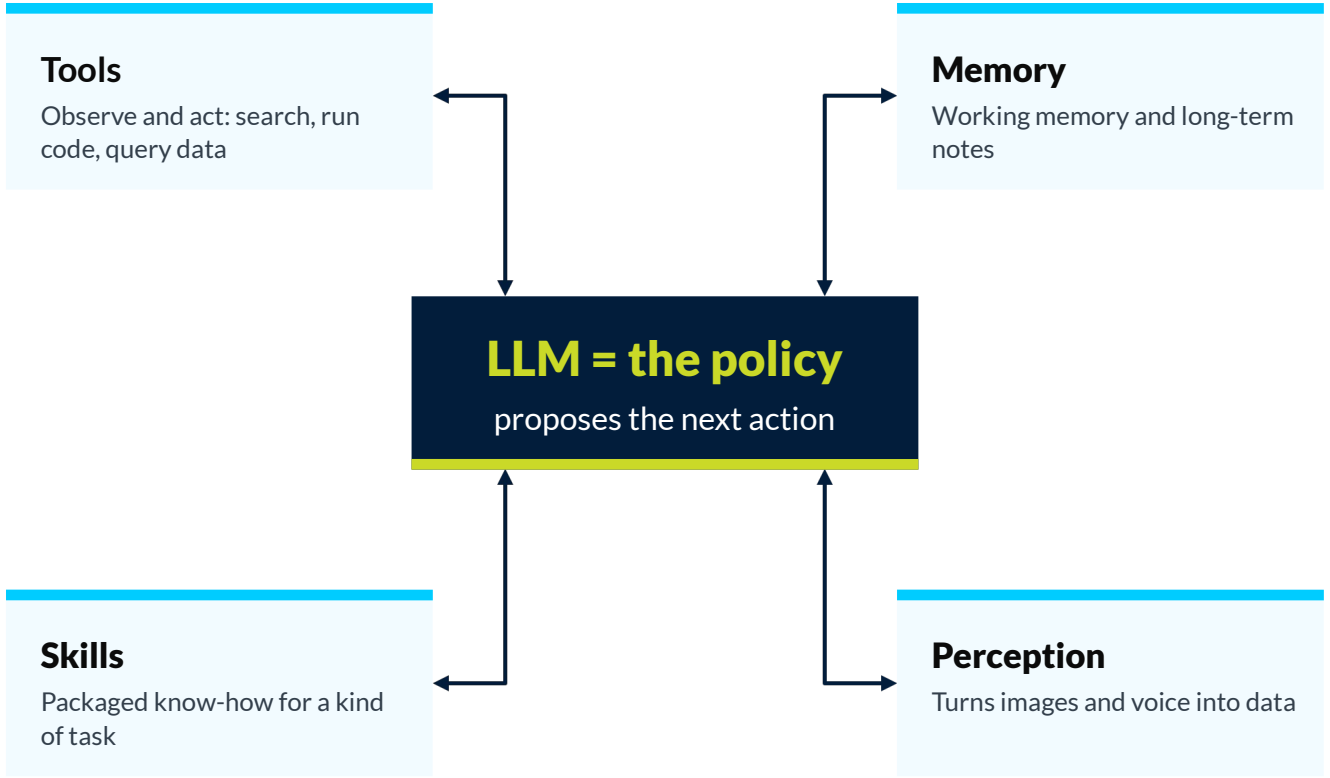
A TRACE (ILLUSTRATIVE)

```
Thought: first, see what fails.
Action: run_tests("test_dates.py")
Observe: 3 failed: wrong time zone
Thought: the parser drops UTC offsets.
Action: edit_file("dates.py")
Action: run_tests()
Observe: all 12 tests pass
Thought: done. Summarise the fix.
```

One loop: reason, act, observe, until the goal is met or a stop rule fires.

Inside a Modern Agent

The LLM proposes one action at a time: answer, call a tool, delegate, ask, stop.



MCP

Connects an agent to **tools and data**

A2A

Connects agents to **other agents**

⚠ **More agents ≠ better:** more calls, more cost, shared blind spots.

Take-Home Task

Design a trip-planning agent

- A 3-day city trip: fixed budget, nothing before 10:00, user's preferences
- Use the small fixed dataset in the brief, not live websites

Hand in before Session 2

- A one-page design sketch: agents, tools, checks, stop rules
- Short answers to the three questions in the brief

Then build a first version

- n8n Community Edition + Ollama (next two slides)
- A broken first version is fine. Bring it.

IT MUST…

1. **Call a tool and use the result**
   The agent cycle
2. **Check the budget in code, not the LLM**
   Hard constraints
3. **Respect the user's preferences**
   Expected utility
4. **Revise at most twice, then hand to a human**
   Stop rules
5. **Ask only if the answer changes the plan**
   Value of information
6. **Log every step it takes**
   Seeing what it did

Every requirement is an idea from today.

Set Up n8n Community Edition

1 **Install Docker Desktop**

From docker.com (Windows, macOS, Linux). Start it before step 2.

2 **Create a volume for your work**

docker volume create n8n_data keeps your workflows when n8n restarts.

3 **Start n8n**

Paste the command on the right into a terminal. Leave that window open.

4 **Open http://localhost:5678**

Create the owner account: an email and a password. It stays on your laptop.

5 **Optional: a free licence key**

Free registration unlocks folders and editor debugging.

TERMINAL

```
# step 2: once
docker volume create n8n_data

# step 3: start n8n
docker run -it --rm --name n8n \
  -p 5678:5678 \
  -e GENERIC_TIMEZONE="Europe/Berlin" \
  -e TZ="Europe/Berlin" \
  -v n8n_data:/home/node/.n8n \
  n8nio/n8n

# no Docker? Node.js 20.19 to 24:
npx n8n
```

Community Edition is the free, self-hosted n8n. No cloud account, no trial.

Gisma University of Applied Sciences

Connect Ollama to n8n

[Take-home · setup 2 of 2, about 10 minutes]

6 Install Ollama

From ollama.com/download. On Linux, use the command on the right.

7 Pull a model that can call tools

ollama pull qwen2.5:7b. Under 8 GB free RAM: **qwen2.5:3b**

8 Add the Ollama credential in n8n

n8n in Docker: **http://host.docker.internal:11434**. With npx: **http://localhost:11434**

9 Smoke test

Chat Trigger → AI Agent, with an Ollama Chat Model attached. Say hello.

TERMINAL

```
# step 6, Linux only
curl -fsSL https://ollama.com/install.sh | sh

# step 7
ollama pull qwen2.5:7b
ollama list
```

IF N8N CAN'T REACH OLLAMA

Is Ollama running? **ollama list** should answer.

With npx: try 127.0.0.1 instead of localhost.

Docker on Linux: add this to docker run:

--add-host=host.docker.internal:host-gateway

Done when: the chat answers, and the execution log shows the Ollama Chat Model ran.

NEXT TIME

In Session 2 we build this agent together: a budget-aware trip planner in n8n + Ollama.

Before then: the take-home task, n8n Community Edition and Ollama.

Thank you! Questions?

Gisma University of Applied Sciences

Photos: Unsplash · Tono Graphy, Cindy F